# Generalized linear models, Bayesian flavors

Frank Edwards

# The components of a model

- Likelihood of the data (the small-world data generator): *e.g.* $y \sim Normal(\mu, \sigma)$
- The linear function (converts likelihood parameters to linear combinations of predictors): *e.g.* $\mu = \alpha + \beta x$
- Priors (our initial beliefs about plausible parameter values) *e.g.*:

$$\alpha \sim Normal(0, 1)$$

$$\beta \sim Normal(0, 0.5)$$

$$\sigma \sim Exp(1)$$

# Limits of the Normal model

There's no reason we must be constrained to the Normal (Gaussian) likelihood for data:

$$y \sim Normal(\mu, \sigma)$$

While powerful, the Normal distribution forces us into assumptions that may not suit our data.

*Maximum entropy* can guide likelihood selection. Which model meets the constraints of the data while maximizing the number of ways a distribution could have arisen?

# Limits of the Normal model

What if our outcome is binary, and we wish to model probability?

- Nothing constrains a model to the [0,1] probability scale

# Limits of the Normal model

What if our outcome is an event count?

- The continuous normal likelihood will predict on a real number scale, inappropriate for non-negative integers (births, deaths, books read, unemployment filings, infections, etc)

# Limits of the Normal model

What if our outcome is categorical?

- The continuous normal model doesn't easily model categorical outcomes (favorite brand of soda, choice of college major)

# Generalizing the linear model

A likelihood can take the following general form

$$\text{outcome} \sim \text{Probability distribution}(\text{parameter})$$

But we generally want to include predictor variables in our model that may help to explain variation in the outcome. That's why we include a *linear predictor* with the following general form

$$\text{parameter} = \text{intercept} + \text{slope} \times \text{predictor}$$

# Mapping a linear predictor to any parameter scale

What if we want to map our linear predictions to some other scale like probability [0,1]?

# Mapping a linear predictor to any parameter scale

What if we want to map our linear predictions to some other scale like probability [0,1]?

# The link function

We can use a link function $g$ to transform our linear predictor from a continuous linear scale to any other scale. The general form is:

$$g(\text{parameter}) = \text{intercept} + \text{slope} \times \text{predictor}$$

# The link function

We can use a link function $g$ to transform our linear predictor from a continuous linear scale to any other scale. The general form is:

$$g(\text{parameter}) = \text{intercept} + \text{slope} \times \text{predictor}$$

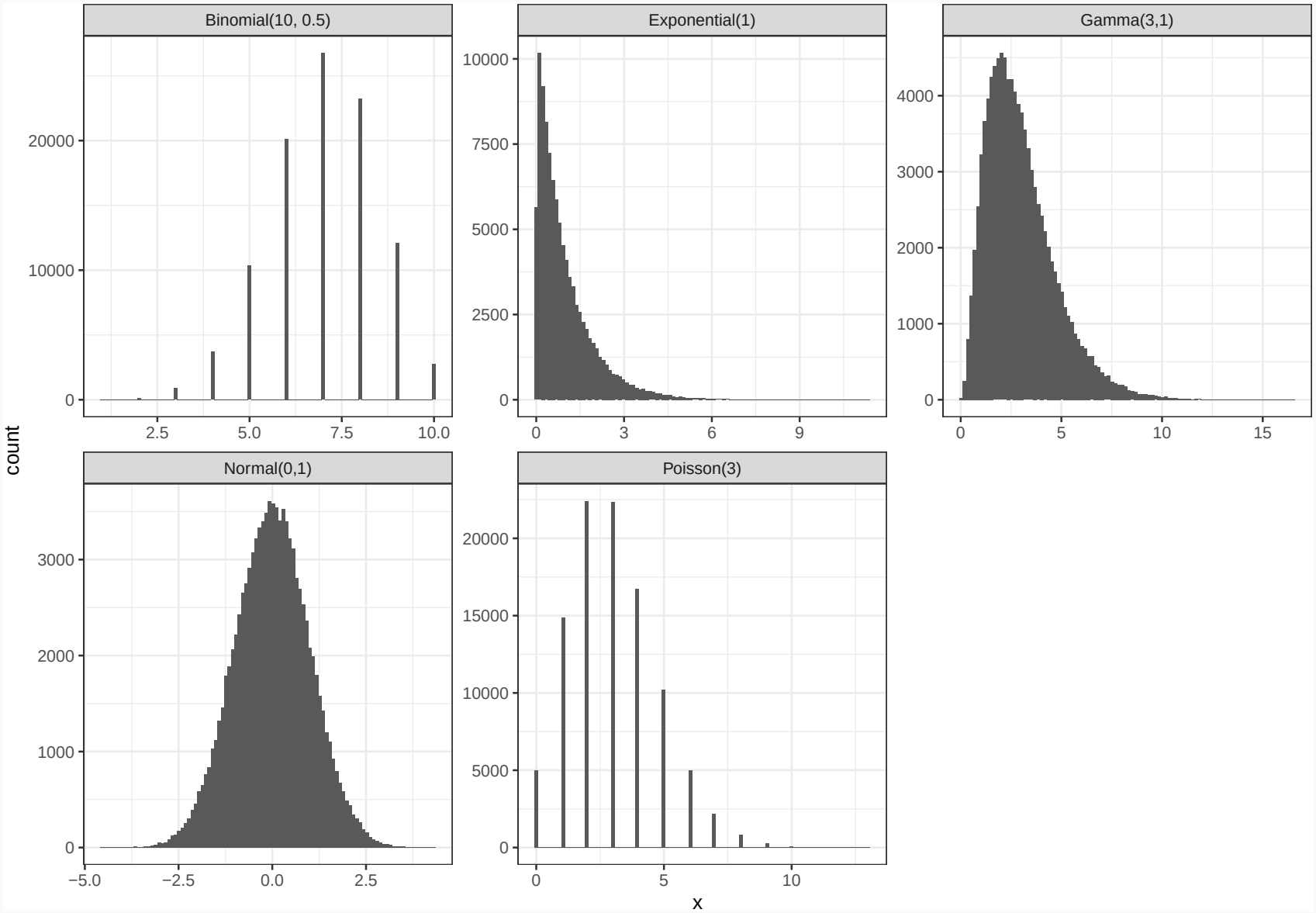
or:

$$\text{parameter} = g^{-1}(\text{intercept} + \text{slope} \times \text{predictor})$$

# Diverse likelihood functions

- Continuous outcome: the Normal distribution
- Continuous non-negative outcome: the exponential distribution
- Continuous non-negative outcome with mode above zero: the Gamma distribution
- Binary outcome: the binomial distribution
- Categorical outcome: the multinomial distribution
- Count outcome: the Poisson distribution

# Common probability distributions used in GLMs



# Read in grad school admissions data

```
admissions <- read_csv("https://stats.idre.ucla.edu/stat/data/binary.csv")
head(admissions)
```

```
# A tibble: 6 x 4
  admit   gre   gpa  rank
  <dbl> <dbl> <dbl> <dbl>
1     0   380  3.61     3
2     1   660  3.67     3
3     1   800  4        1
4     1   640  3.19     4
5     0   520  2.93     4
6     1   760  3        2
```

```
nrow(admissions)
```

```
[1] 400
```

# Evaluate distribution of binary admission variable

```
admissions |>
  group_by(admit) |>
  summarize(n = n()) |>
  mutate(prop = n/sum(n))
```

```
# A tibble: 2 x 3
  admit     n  prop
  <dbl> <int> <dbl>
1     0   273 0.682
2     1   127 0.318
```

# How do GRE scores relate to admission?

Let's consider the linear regression model, where GRE_s is a mean-centered SD scaled GRE score, so a 1 is one unit above the mean:

$$admit_i \sim Normal(\mu, \sigma)$$

$$\mu = \alpha + \beta \times \text{GRE}$$

$$\alpha \sim N(0.5, 1)$$

$$\beta \sim N(0, 1)$$

$$\sigma \sim Exp(1)$$

Because this is a Normal likelihood with a binary outcome, we call it a *linear probability model*.

# Fit the model using MCMC

```
### DON'T use PERIODS in VARIABLE names for Stan
### I have done it 5 times already...
d<-admissions |>
  mutate(gre_s = scale(gre)) |>
  select(admit, gre_s)

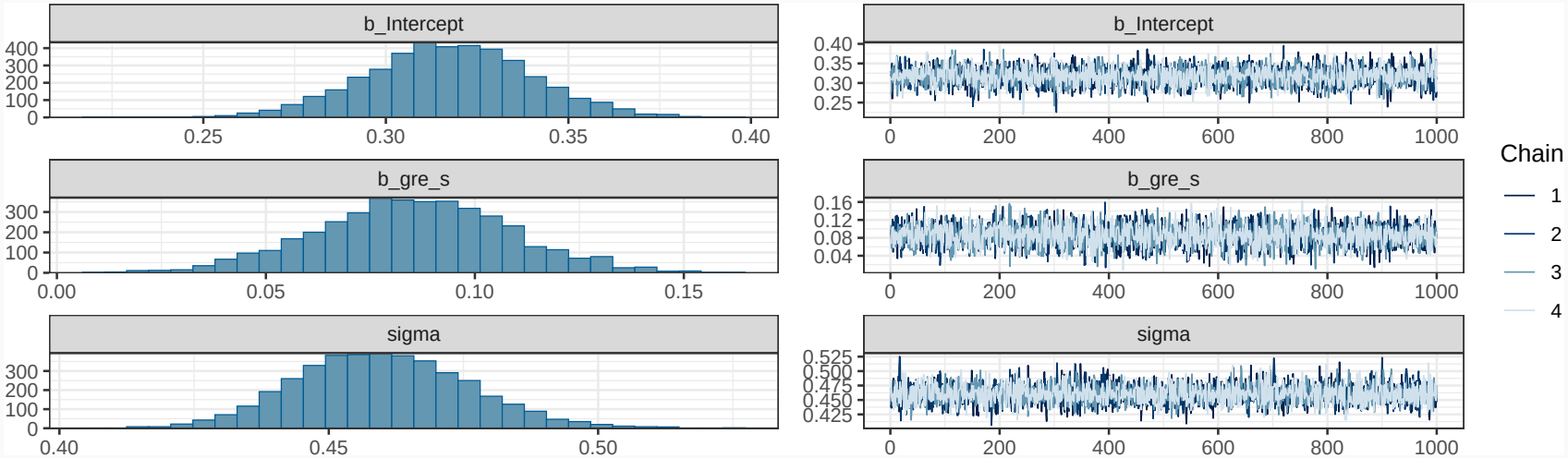
d<-as.data.frame(d)

priors <- c(set_prior("normal(0.5, 1)", class = "Intercept"),
            set_prior("normal(0,1)", class = "b"),
            set_prior("exponential(1)", class = "sigma"))

m0 <- brm(admit ~ gre_s,
          data = d,
          prior = priors,
          cores = 4)
```

# Checking convergence diagnostics

```
plot(m0)
```



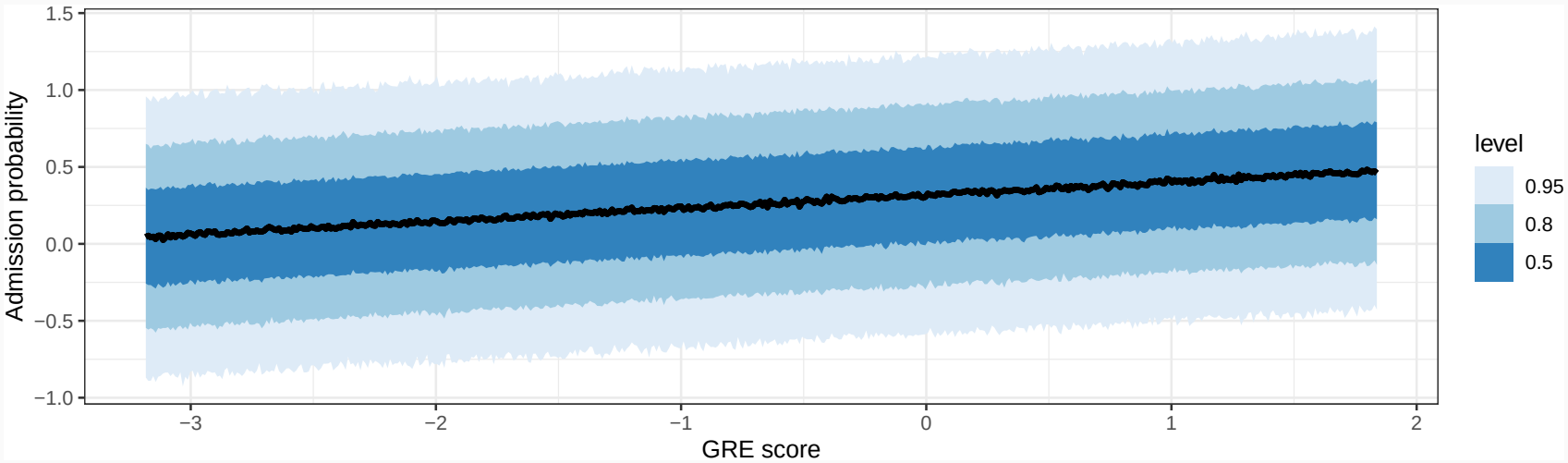
# Evaluating the fit

Let's predict admissions probabilities for the full range of GRE scores.

```
## set up counterfactual scenarios
sim_dat<-data.frame(gre_s =
                      seq(min(d$gre_s),
                          max(d$gre_s),
                          length.out = 500))
## draw from posterior predictive distribution
sim_dat <- sim_dat |>
  add_predicted_draws(m0)
```

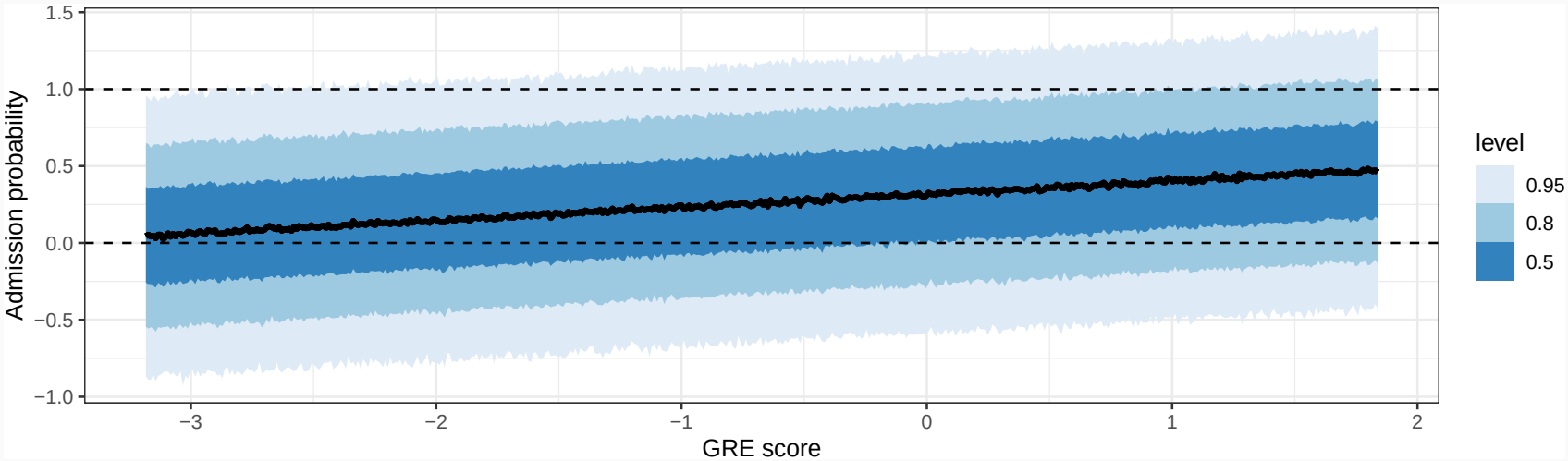
# Posterior predictions

```
ggplot(sim_dat,
       aes(x = gre_s, y = .prediction)) +
  stat_lineribbon() +
  scale_fill_brewer() +
  labs(y = "Admission probability", x = "GRE score")
```



# One of the limits of the linear probability model

```
ggplot(sim_dat,
       aes(x = gre_s, y = .prediction)) +
  stat_lineribbon() +
  scale_fill_brewer() +
  geom_hline(yintercept = 0, lty = 2) +
  geom_hline(yintercept = 1, lty = 2) +
  labs(y = "Admission probability", x = "GRE score")
```



# The Bernoulli distribution

Binary outcomes can be described easily by the Bernoulli distribution

$$y \sim Bernoulli(p)$$

$$\Pr(y = 1) = p = 1 - \Pr(y = 0)$$

$$E(y) = \bar{y} = p$$

$$Var(y) = p(1 - p)$$

Bernoulli variables are a special case of the Binomial distribution, with $n = 1$

# What is E(admit) and var(admit)

```
### E(y) = p = sum(y==1)/n
mean(admissions$admit)
```

```
[1] 0.3175
```

```
### var(y) = p(1-p)
var(admissions$admit)
```

```
[1] 0.2172368
```

```
mean(admissions$admit) * (1 - mean(admissions$admit))
```

```
[1] 0.2166938
```

# Let's treat admission as a Bernoulli / Binomial random variable

Because admission is a binary outcome, ["admitted", "not admitted"], a Bernoulli (equivalent to Binomial with n=1) model is the most appropriate. It is the distribution that satisfies the constraints of the data generating process, while also satisfying maximum entropy.

$$\text{admit}_i \sim Binomial(n = 1, p_i)$$

# Let's treat admission as a Bernoulli / Binomial random variable

Because admission is a binary outcome, ["admitted", "not admitted"], a Bernoulli (equivalent to Binomial with n=1) model is the most appropriate. It is the distribution that satisfies the constraints of the data generating process, while also satisfying maximum entropy.

$$\text{admit}_i \sim Binomial(n = 1, p_i)$$

We'll use a linear predictor with link function $g$

$$g(p_i) = \alpha + \beta \times \text{GRE}_i$$

# Let's try a different approach

This Binomial distribution has one unobserved parameter, $p$. The parameter $p$ is the probability of "success". So we need a link function that translates a continuous linear function onto a [0,1] probability space.

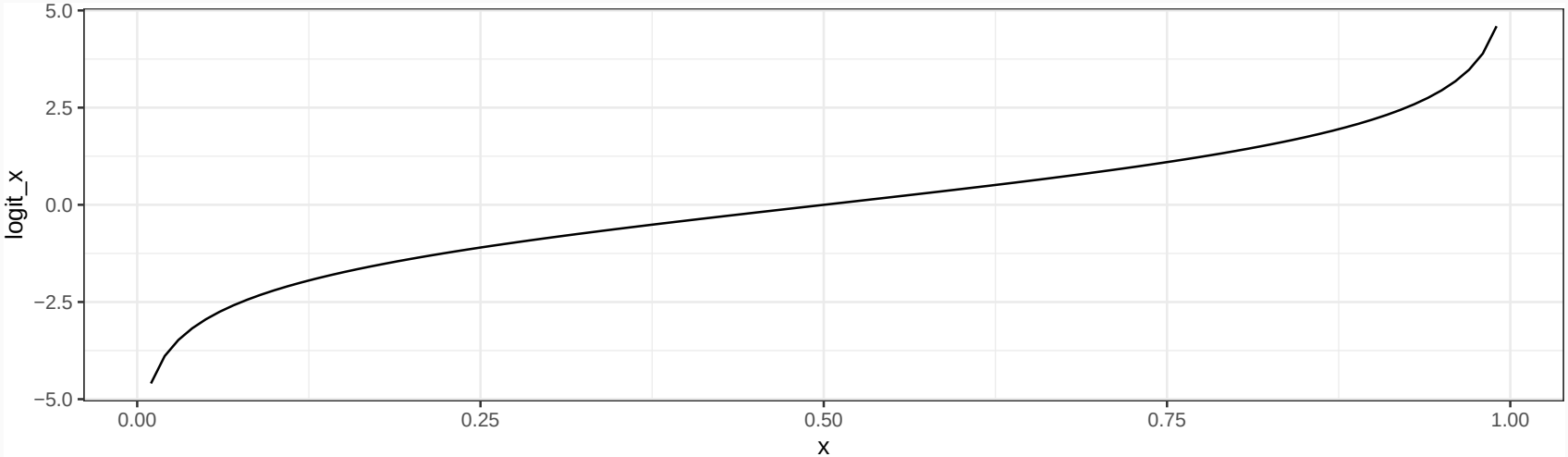
The logit function is constrained to [0,1], and serves this job well. Along with log, the logit is a very common link function for GLMs.

$$logit(p) = log\frac{p}{1-p}$$

# The logit function

To translate a $[0, 1]$ probability to a continuous scale, we use the logit function: $logit(p) = log(\frac{p}{1-p})$

```
x<-seq(0.01, 0.99, 0.01)
logit_x = log(x/(1-x))
plot_dat<-data.frame(x = x, logit_x = logit_x)
ggplot(plot_dat, aes(x = x, y = logit_x)) +
  geom_line()
```

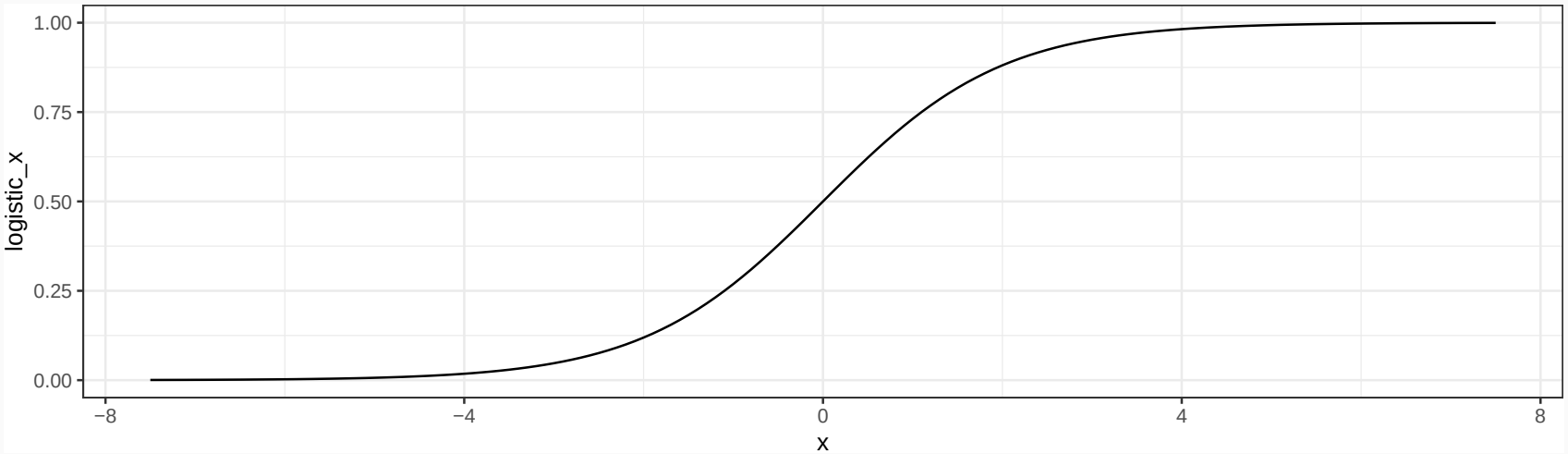


# The inverse logit (logistic) function

To translate the linear predictor $[-\infty, \infty]$ to the probability $[0, 1]$ scale, we can use the inverse logit function (also called the logistic function)

$$logit^{-1}(x) = \frac{1}{1 + exp(-x)}$$

```
x<-seq(-7.5, 7.5, 0.1)
logistic_x<-1/(1 + exp(-x))
plot_dat<-data.frame(x = x, logistic_x = logistic_x)
ggplot(plot_dat, aes(x = x, y = logistic_x)) +
  geom_line()
```



# A generalized linear model

We use a logit link function to move back and forth between a linear (logit) scale, and a probability (outcome) scale. I'll use weakly informative priors.

$$\text{admit}_i \sim Binomial(1, p_i)$$

$$logit(p_i) = \alpha + \beta \times \text{GRE}$$

$$\alpha \sim N(0, 2)$$

$$\beta \sim N(0, 2)$$

# Estimating the model

```
priors <- c(set_prior("normal(0,2)", class = "Intercept"),
            set_prior("normal(0,2)", class = "b"))

m1 <- brm(admit~gre_s,
          data = d,
          prior = priors,
          family = "bernoulli",
          cores = 4)
```

# Logistic regression parameter estimates

Remember that our linear predictor is now on the *logit* scale. This makes interpretation hard!

```
summary(m1)
```

```
 Family: bernoulli
  Links: mu = logit
Formula: admit ~ gre_s
   Data: d (Number of observations: 400)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

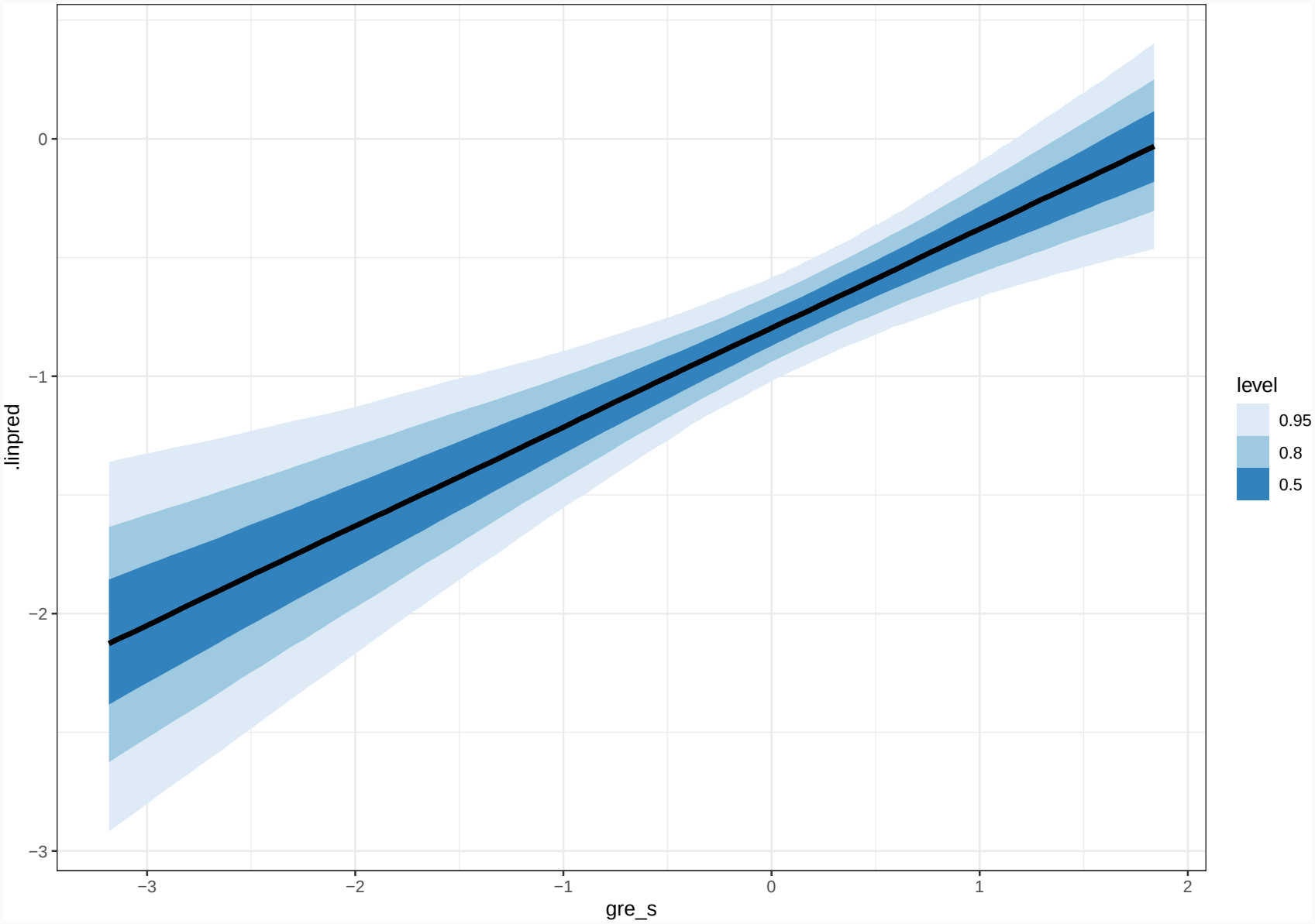
Regression Coefficients:
          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept    -0.80      0.11    -1.02    -0.58 1.00     3082     2706
gre_s         0.42      0.11     0.19     0.64 1.00     3109     2551

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

# Visualizing the impact of GRE scores

**General advice for GLMs:** Avoid over-interpreting parameter estimates, they are now non-linear and multiplicative. Use the inverse-link functions to visualize relationships on the scale of the outcome (e.g. probability).

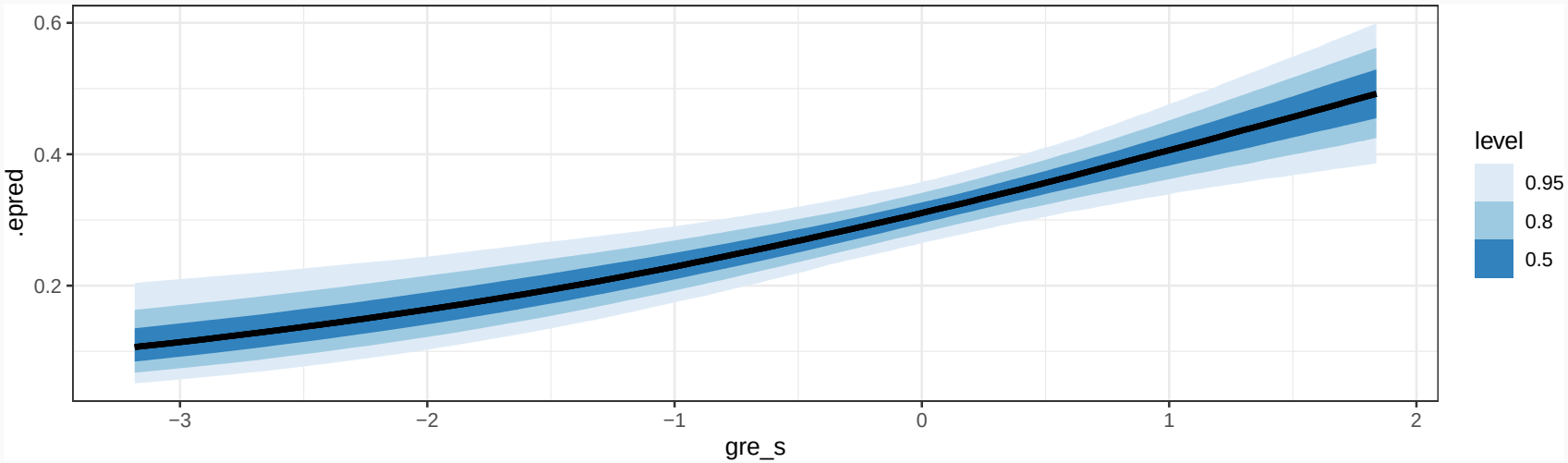
# On the logit scale



# On the probability scale

```
### define GRE scores over the range of observed
sim_dat<-data.frame(gre_s = seq(
  min(d$gre_s), max(d$gre_s), length.out = 500))
### epred applies the inverse link function
sim_dat <- sim_dat |>
  add_epred_draws(m1)

ggplot(sim_dat, aes(x = gre_s,
                    y = .epred)) +
  stat_lineribbon() +
  scale_fill_brewer()
```



# Extending the binomial model

A logistic regression is a binomial regression with the special context that the parameter $n$ is fixed at 1. It estimates individual level event probabilities.

Let's load different grad school admissions data from rethinking. We want to know whether female applicants are less likely to be admitted than male applicants.

```
data("UCBadmit")
d<-UCBadmit
```

# Count data

```
head(d)
```

```
  dept applicant.gender admit reject applications
1    A             male   512    313          825
2    A           female    89     19          108
3    B             male   353    207          560
4    B           female    17      8           25
5    C             male   120    205          325
6    C           female   202    391          593
```

We don't have individual admission data. These are individual-level data, aggregated to the department level by applicant gender, counts of individual outcomes.

# Translating counts into proportions

```
d |>
  group_by(applicant.gender) |>
  summarise(admit_prop = sum(admit) / sum(applications))
```

```
# A tibble: 2 x 2
  applicant.gender admit_prop
  <fct>                  <dbl>
1 female                 0.304
2 male                   0.445
```

# Defining a model

The count of admissions in department $i$ will be modeled with two likelihood parameters: the number of applicants, $n$, and the probability of admission $p$. We will use a logit link to define a linear function for $p$ to include applicant gender as a predictor of admission rates.

$$admit_i \sim Binomial(n_i, p_i)$$

$$logit(p_i) = \alpha_{gender[i]}$$

$$\alpha \sim N(0, 1.5)$$

On the logit scale, $N(0, 1.5)$ is a weakly informative prior centered on p = 0.5 with substantial variance.

# Estimating the model

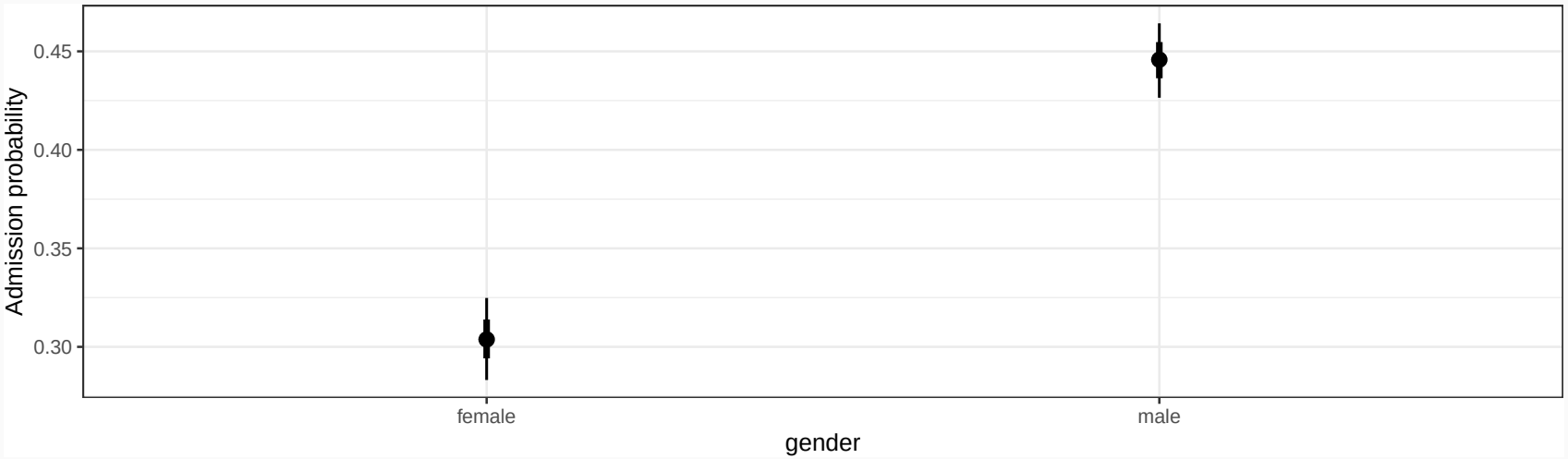
```
d_slim <- d |>
  mutate(gender = (applicant.gender == "female") + 1) |>
  select(gender, admit, applications)

priors <- c(set_prior("normal(0,1.5)", class = "b"))

m2 <- brm(admit|trials(applications) ~ -1 + factor(gender),
          data = d_slim,
          family = "binomial",
          prior = priors,
          core = 4)
```

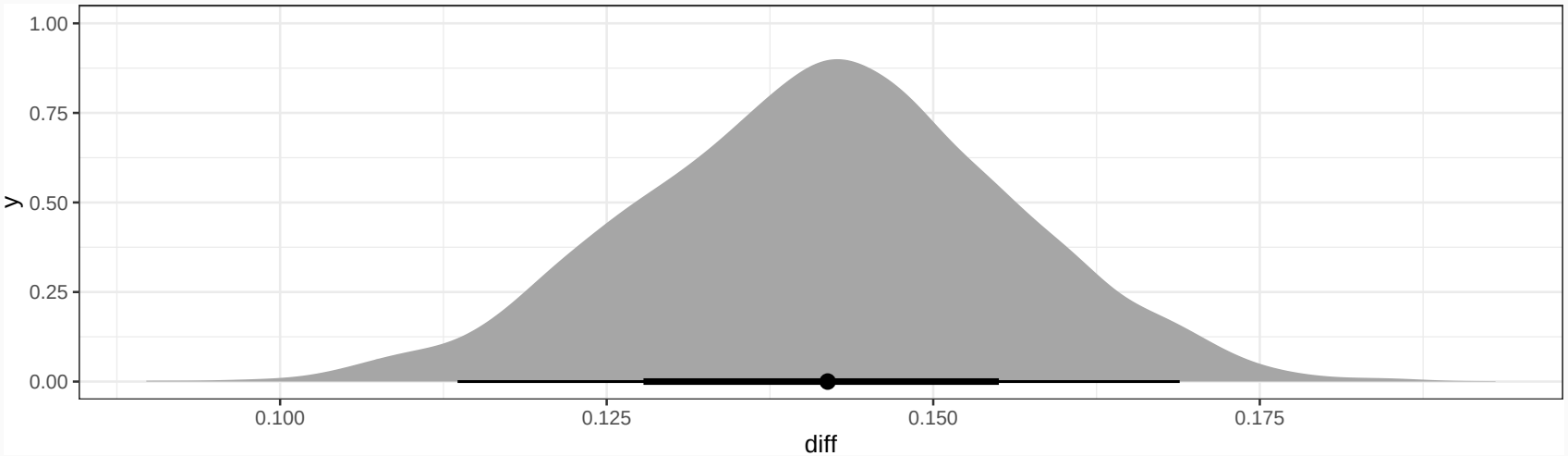
# Interpreting the model: probability

```
sim_dat<-data.frame(gender = c(1, 2), applications = 1)
sim_dat <- sim_dat |>
  add_epred_draws(m2)
sim_dat<-sim_dat |>
  mutate(gender = ifelse(gender == 2, "female", "male"))
ggplot(sim_dat, aes(x = gender, y = .epred)) +
  stat_pointinterval() +
  labs(y = "Admission probability")
```



# Interpreting the model: difference in probability

```
pred_m <- sim_dat |>
  filter(gender=="male")
pred_f <- sim_dat |>
  filter(gender=="female")
# compute contrast
plot_dat<-data.frame(diff = pred_m$.epred - pred_f$.epred)
ggplot(plot_dat, aes(x = diff)) +
  stat_slabinterval()
```



# Perhaps selecting to apply to more competitive programs plays a role?

$$admit_i \sim Binomial(n_i, p_i)$$

$$logit(p_i) = \alpha_{gender[i]} + \delta_{program[i]}$$

$$\alpha_{gender} \sim N(0, 1.5)$$

$$\delta_{program} \sim N(0, 1.5)$$

# Estimating the model

```
d_slim<-d |>
  mutate(gender = as.numeric(applicant.gender),
         dept = as.numeric(dept)) |>
  select(gender, admit, applications, dept)

m3 <- brm(admit | trials(applications) ~ -1 + factor(gender) + factor(dept),
          family = "binomial",
          prior = priors,
          data = d_slim,
          cores = 4)
```

# Model results

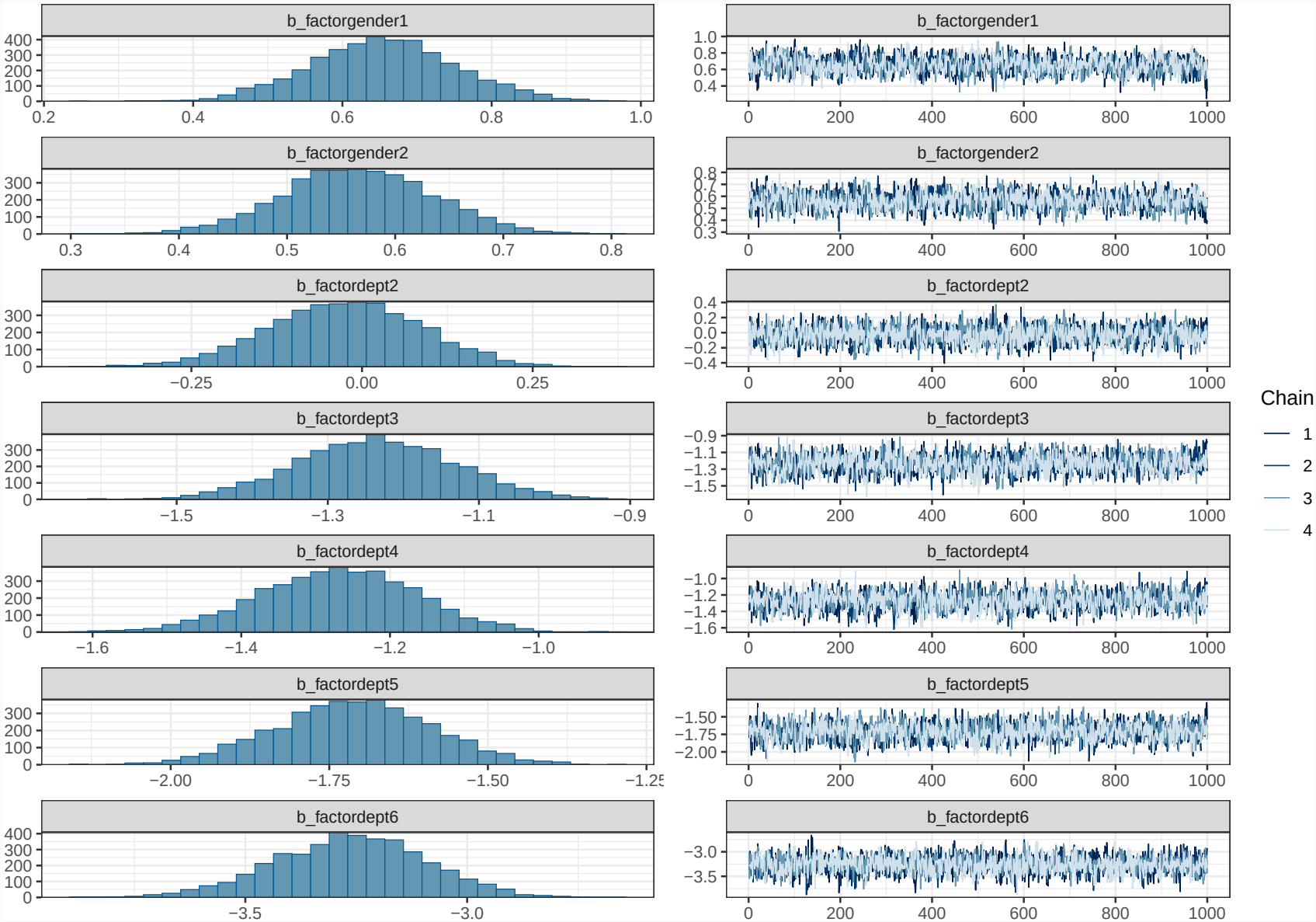
```
summary(m3)
```

```
 Family: binomial
  Links: mu = logit
Formula: admit | trials(applications) ~ -1 + factor(gender) + factor(dept)
   Data: d_slim (Number of observations: 12)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
              Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
factorgender1     0.66      0.10     0.47     0.85 1.01     1273     1950
factorgender2     0.57      0.07     0.43     0.70 1.01     1429     1998
factordept2      -0.02      0.11    -0.25     0.19 1.00     1967     2761
factordept3      -1.24      0.11    -1.44    -1.03 1.01     1576     2258
factordept4      -1.27      0.11    -1.48    -1.07 1.00     1602     2071
factordept5      -1.71      0.12    -1.96    -1.47 1.01     1760     2427
factordept6      -3.26      0.17    -3.61    -2.94 1.00     2522     2281

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

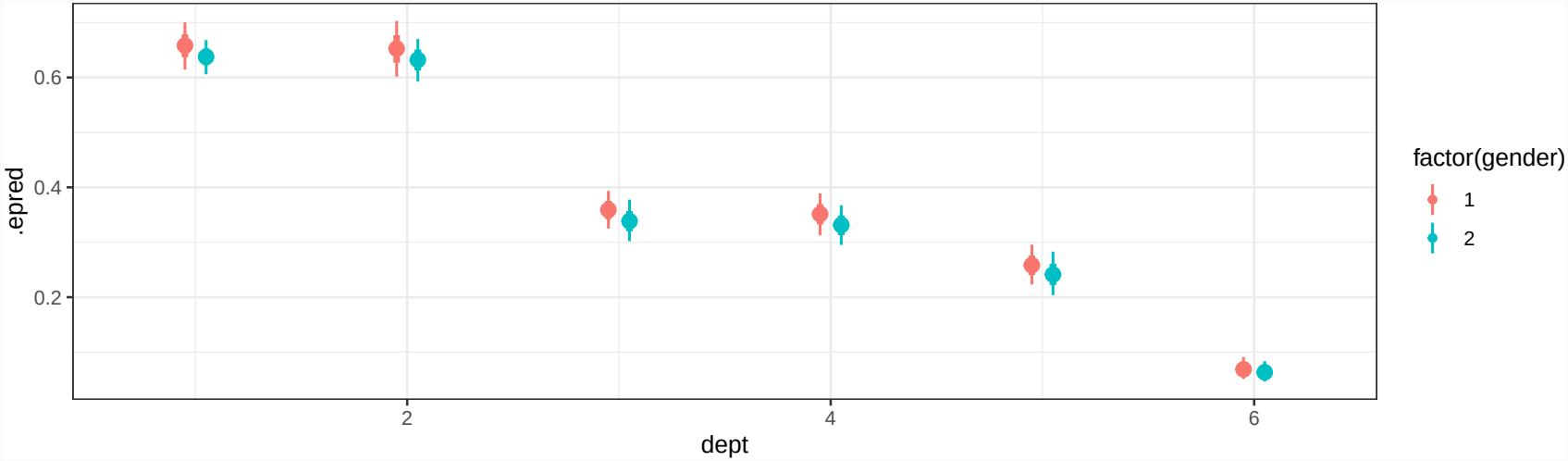
# Checking convergence



# Visualizing the results

```
sim_dat<-expand_grid(gender = unique(d_slim$gender),
                     dept = unique(d_slim$dept),
                     applications = 1)
## add expected values
sim_dat <- sim_dat |>
  add_epred_draws(m3)

ggplot(sim_dat, aes(x = dept, color = factor(gender),
                    y =.epred)) +
  stat_pointinterval(position = position_dodge(width=0.2))
```



# The Binomial count model

When $n = 1$, a Binomial regression model with a logit link function is called a logistic regression. When $n > 1$, we are instead modeling counts of events as our outcome.

The expected value for a Binomial variable with probability $p$ and number of trials $n$ is $np$, and the variance is $np(1 - p)$.

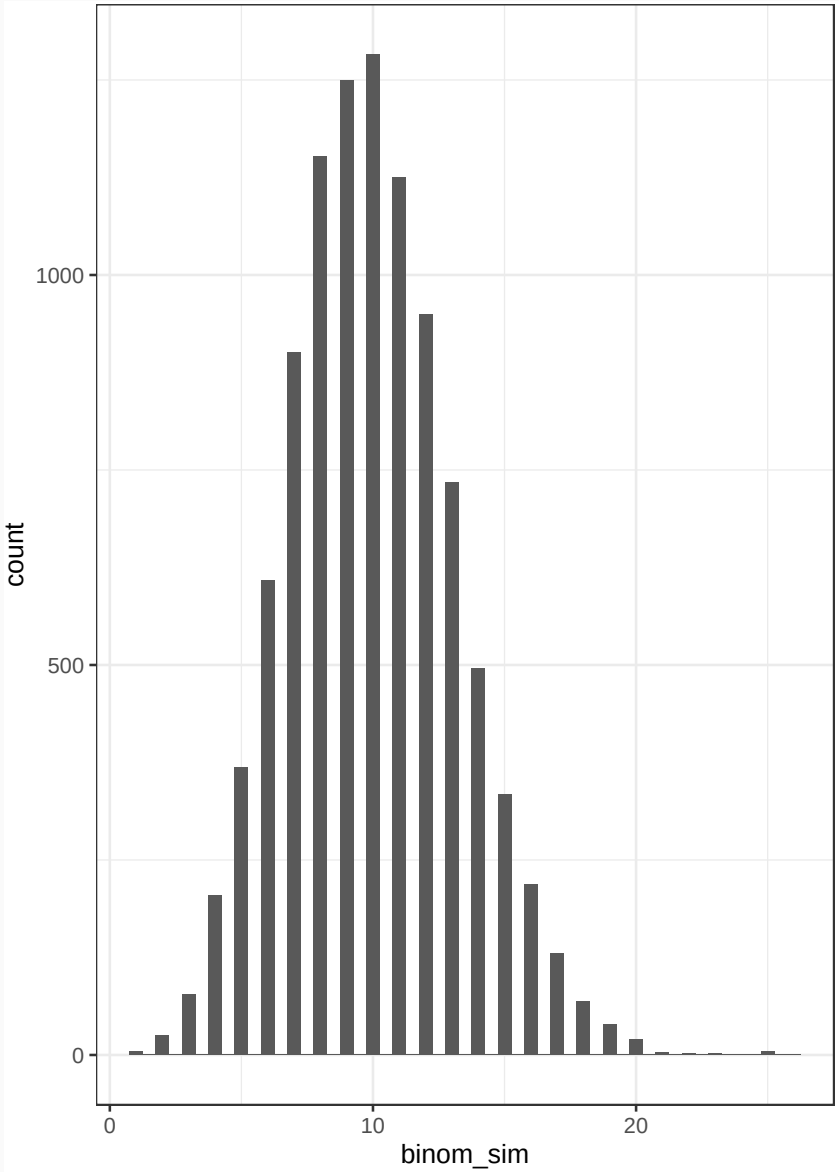
# The Binomial count model

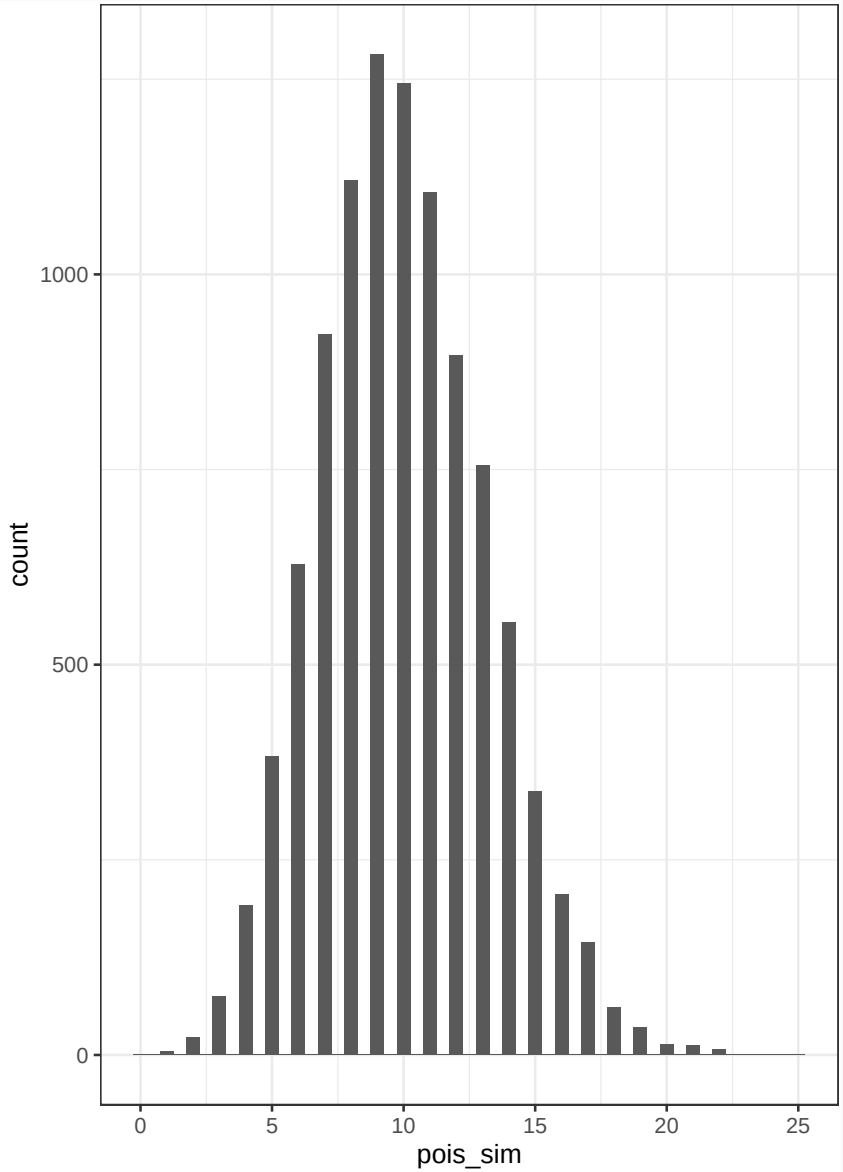
When $n = 1$, a Binomial regression model with a logit link function is called a logistic regression. When $n > 1$, we are instead modeling counts of events as our outcome.

The expected value for a Binomial variable with probability $p$ and number of trials $n$ is $np$, and the variance is $np(1-p)$.

When $n$ is large, and $p$ is small, $np \approx np(1-p)$. This special case of the Binomial distribution is the Poisson distribution.

# Convergence



# The Poisson likelihood

Where y is a non-negative integer (count)

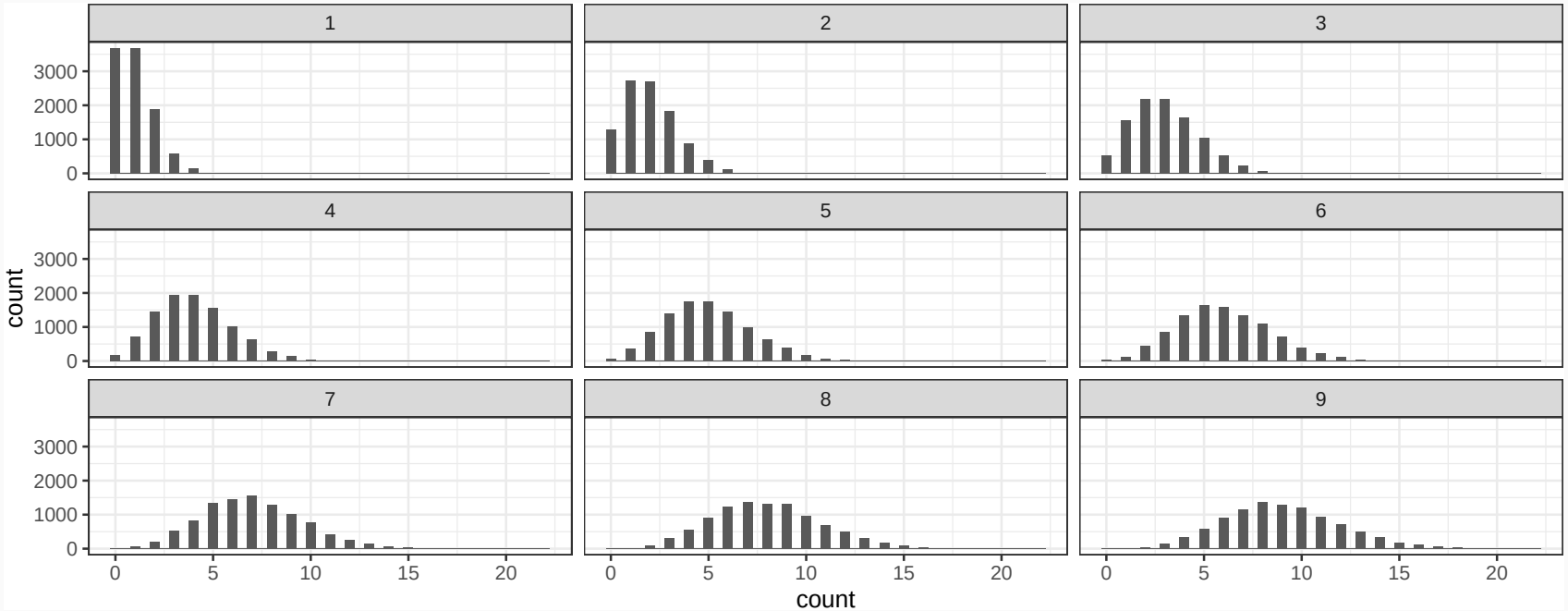
$$y \sim Poisson(\lambda)$$

$$E(y) = \bar{y} = \lambda$$

$$Var(y) = \lambda$$

# The Poisson likelihood

```
ggplot(pois_demo, aes(x=count)) +
  geom_histogram(binwidth = 0.5) +
  facet_wrap(~lambda)
```



# Let's look at each Poisson variable

```
pois_demo|>group_by(lambda)|>

  summarise(mean = mean(count),
            variance = var(count))
```

```
# A tibble: 9 x 3
  lambda  mean variance
   <int> <dbl>    <dbl>
1      1 0.994    0.982
2      2 2.02     2.00
3      3 2.98     3.03
4      4 4.00     4.02
5      5 4.97     5.02
6      6 6.04     5.96
7      7 7.01     6.85
8      8 8.03     8.03
9      9 9.02     9.06
```

# Poisson models as a GLM

For a non-negative integer $y$

$$y_i \sim Poisson(\lambda)$$

$$\log \lambda_i = \alpha + \beta X$$

# New data: Fatal Encounters

Let's use data on police-involved killings to estimate how many men of different racial groups are killed by police in an average US county.

```
fe<-read_csv("./fe_demo.csv")
### on your computer, run fe<-read_csv("./slides/fe_demo.csv")
head(fe)
```

```
# A tibble: 6 x 5
   fips state   pop deaths race.ethn
  <dbl> <chr> <dbl>  <dbl> <chr>
1  1001 AL      483      0 latino
2  1003 AL     3218      0 latino
3  1005 AL      473      0 latino
4  1007 AL      163      0 latino
5  1009 AL     1713      0 latino
6  1011 AL      359      0 latino
```

# A mortality model using a Poisson likelihood

Let's define a Poisson regression of county-level death rates $d$ for each racial group

$$d_i \sim Poisson(\lambda_i)$$

$$\log \lambda_i = \alpha[race]_i$$

$$\alpha \sim Normal(0, 2)$$

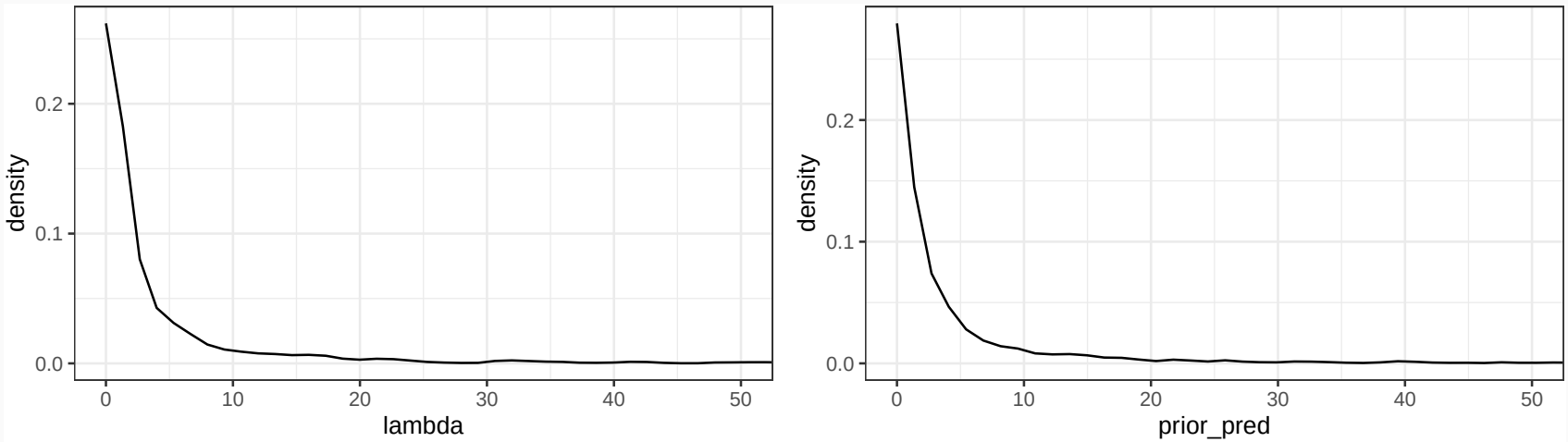
# Prior simulation

```r
plot_dat <- data.frame(alpha = rnorm(1000, 0, 2))
## apply the inverse link function, e^\alpha = \lambda
plot_dat <- plot_dat |>
  mutate(lambda = exp(alpha),
         prior_pred = rpois(1000, lambda))
head(plot_dat)
```

```
      alpha      lambda prior_pred
1 -1.451164  0.23429744          0
2  2.291714  9.89188128         12
3 -2.306806  0.09957879          0
4 -3.759774  0.02328901          0
5  3.161263 23.60038956         25
6  1.288314  3.62666760          2
```

# Prior and prior predictive distributions

```
p1 <- ggplot(plot_dat,
       aes(x = lambda)) +
  geom_density() +
  coord_cartesian(xlim=c(0, 50))
p2 <- ggplot(plot_dat,
             aes(x = prior_pred)) +
  geom_density() +
  coord_cartesian(xlim=c(0,50))
p1+p2
```



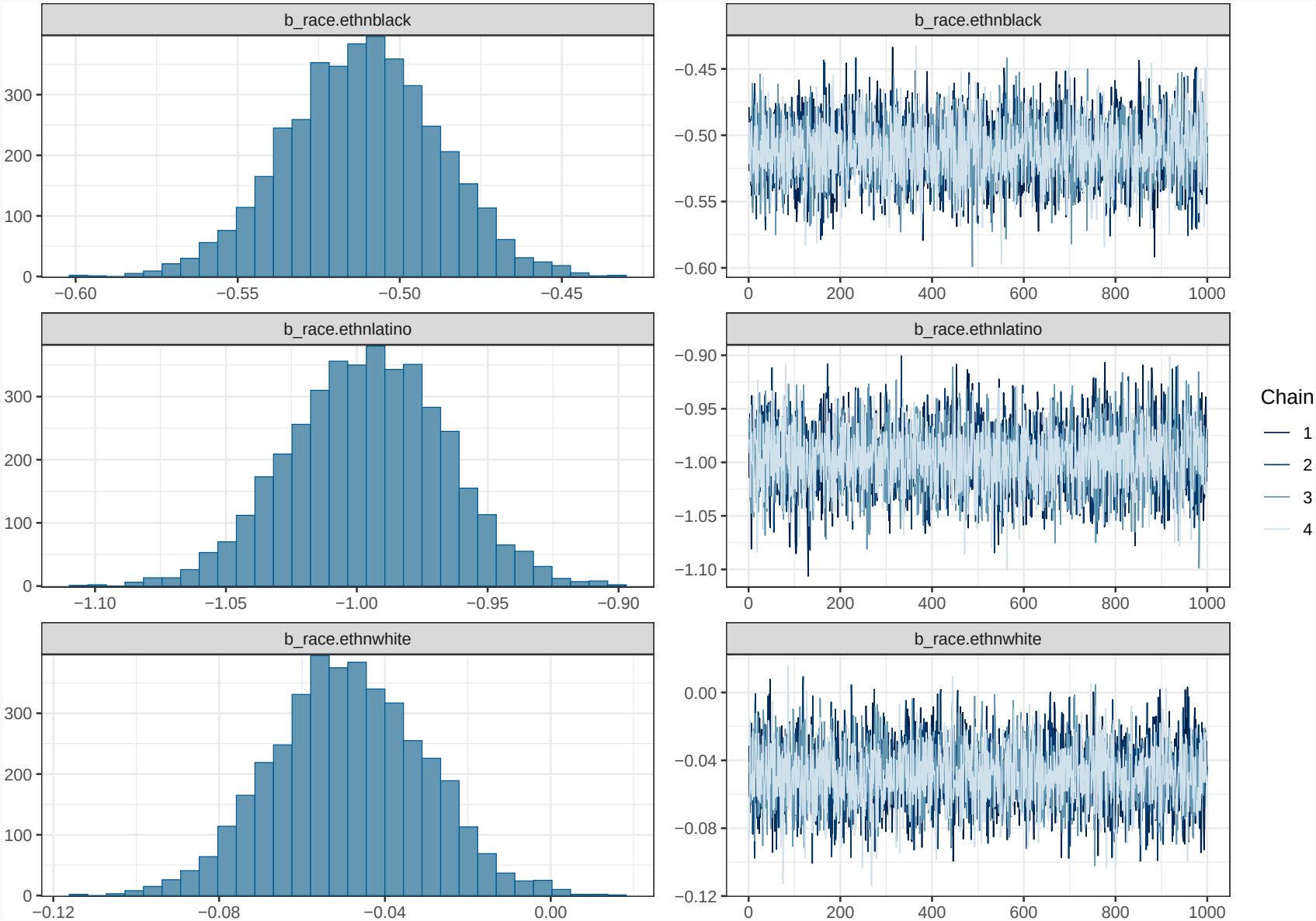
# Estimation

```
# remove counties with zero reported pop for subgroups
fe_model<-fe |>
  filter(pop>0)

priors <- set_prior("normal(0,2)", class = "b")

fe_pois <- brm(deaths ~ -1 + race.ethn,
               data = fe_model,
               family = "poisson",
               prior = priors,
               cores = 4)
```

# Check convergence



# Log-scale parameters are hard to interpret...

```
summary(fe_pois)
```

```
 Family: poisson
  Links: mu = log
Formula: deaths ~ -1 + race.ethn
   Data: fe_model (Number of observations: 9207)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

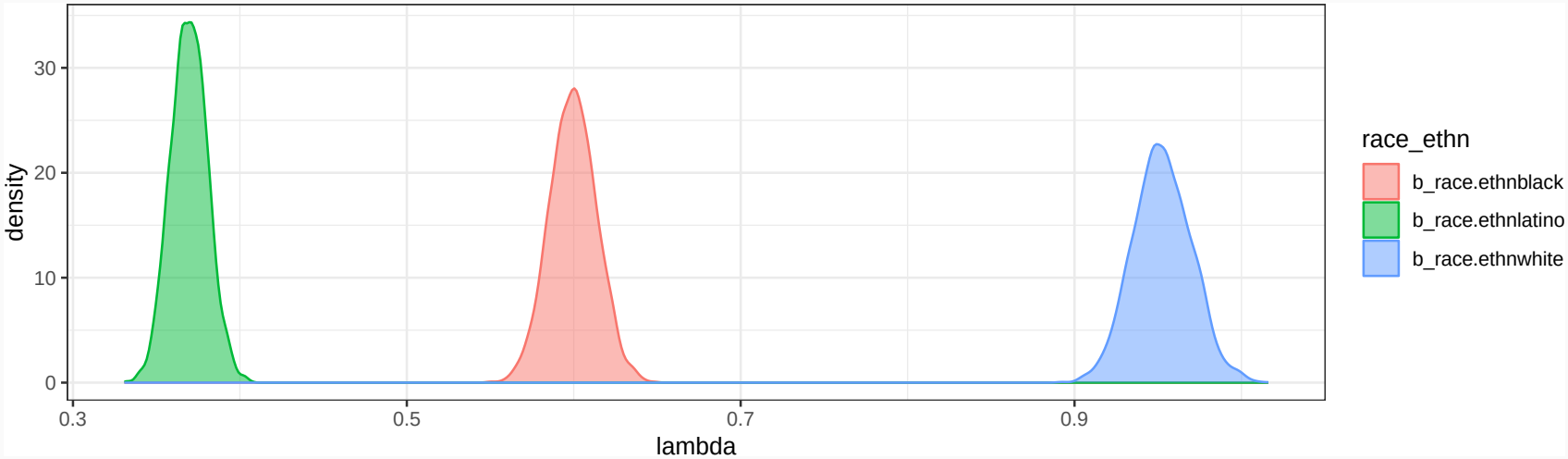
Regression Coefficients:
               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
race.ethnblack    -0.51      0.02    -0.56    -0.47 1.00     4054     2946
race.ethnlatino   -1.00      0.03    -1.05    -0.94 1.00     4338     3003
race.ethnwhite    -0.05      0.02    -0.08    -0.01 1.00     4287     3113

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

# Expected deaths in an average county

```
fe_pois_post <- as_draws_df(fe_pois)
fe_pois_post <- fe_pois_post |>
  pivot_longer(cols = b_race.ethnblack:b_race.ethnwhite,
               names_to = "race_ethn",
               values_to = "log_lambda")
# inverse link function to get lambda on E(deaths) scale
fe_pois_post<-fe_pois_post |>
  mutate(lambda = exp(log_lambda))

ggplot(fe_pois_post,
       aes(x = lambda, fill = race_ethn, color = race_ethn)) +
  geom_density(alpha = 0.5)
```



# Problem: these don't adjust for population size

The standard Poisson model is *unbounded*, and assumes each observation is drawn from a similar unit. However, we can use Poisson models to estimate event *rates* using an *offset*.

# Problem: these don't adjust for population size

The standard Poisson model is *unbounded*, and assumes each observation is drawn from a similar unit. However, we can use Poisson models to estimate event *rates* using an *offset*.

An offset (or exposure) scales $\lambda$ to the size of the exposure, turning $\lambda$ into a rate per unit of the offset.

# Including an offset in the model

Let's now estimate a model that includes population $p$ as an exposure variable (typically time, population, or some other measure of event opportunity).

$$d_i \sim Poisson(\lambda)$$

$$\log \frac{\lambda_i}{p_i} = \log \lambda_i - \log p_i = \alpha + \beta x_i$$

# Including an offset in the model

Let's now estimate a model that includes population $p$ as an exposure variable (typically time, population, or some other measure of event opportunity).

$$d_i \sim Poisson(\lambda)$$

$$\log \frac{\lambda_i}{p_i} = \log \lambda_i - \log p_i = \alpha + \beta x_i$$

Which we can rewrite as

$$\log \lambda_i = \alpha + \beta x_i + \log(p_i)$$

We add the offset to the regression without including a parameter.

# Let's re-estimate the model with an offset

$$d_i \sim Poisson(\lambda)$$

$$\log \lambda_i = \alpha[race]_i + \log p_i$$

$$\alpha[race] \sim Normal(0, 2)$$

```
fe_pois2 <- brm(deaths ~ -1 + race.ethn + offset(log(pop)),
                data = fe_model,
                family = "poisson",
                prior = priors,
                cores = 4)
```

# Model output

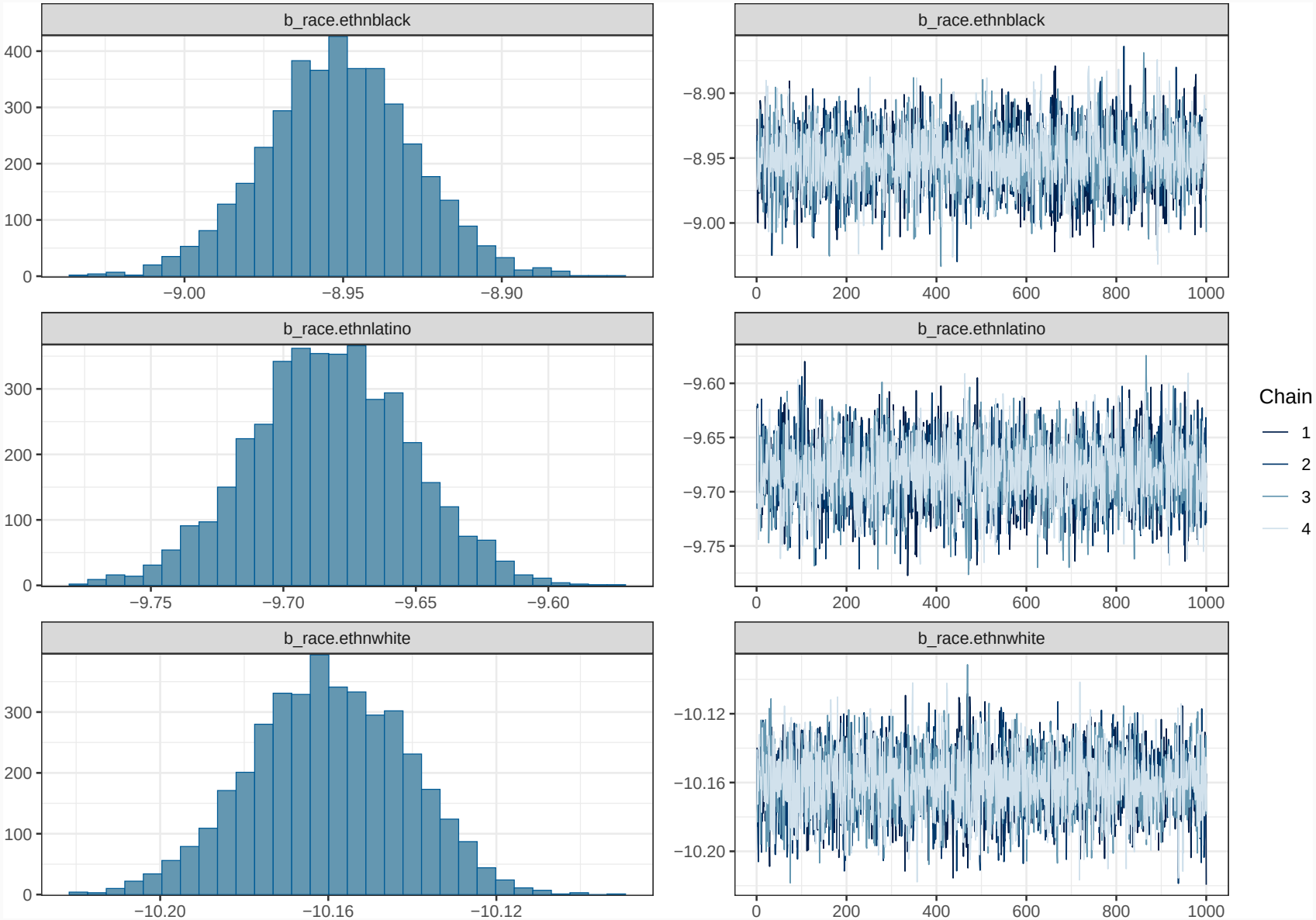
```
summary(fe_pois2)
```

```
 Family: poisson
  Links: mu = log
Formula: deaths ~ -1 + race.ethn + offset(log(pop))
   Data: fe_model (Number of observations: 9207)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
               Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
race.ethnblack     -8.95      0.02    -9.00    -8.91 1.00     4291     2514
race.ethnlatino    -9.68      0.03    -9.74    -9.62 1.00     3841     3094
race.ethnwhite    -10.16      0.02   -10.20   -10.12 1.00     4270     2809

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

# Interpreting the model, checking convergence diagnostics



# Posterior inference

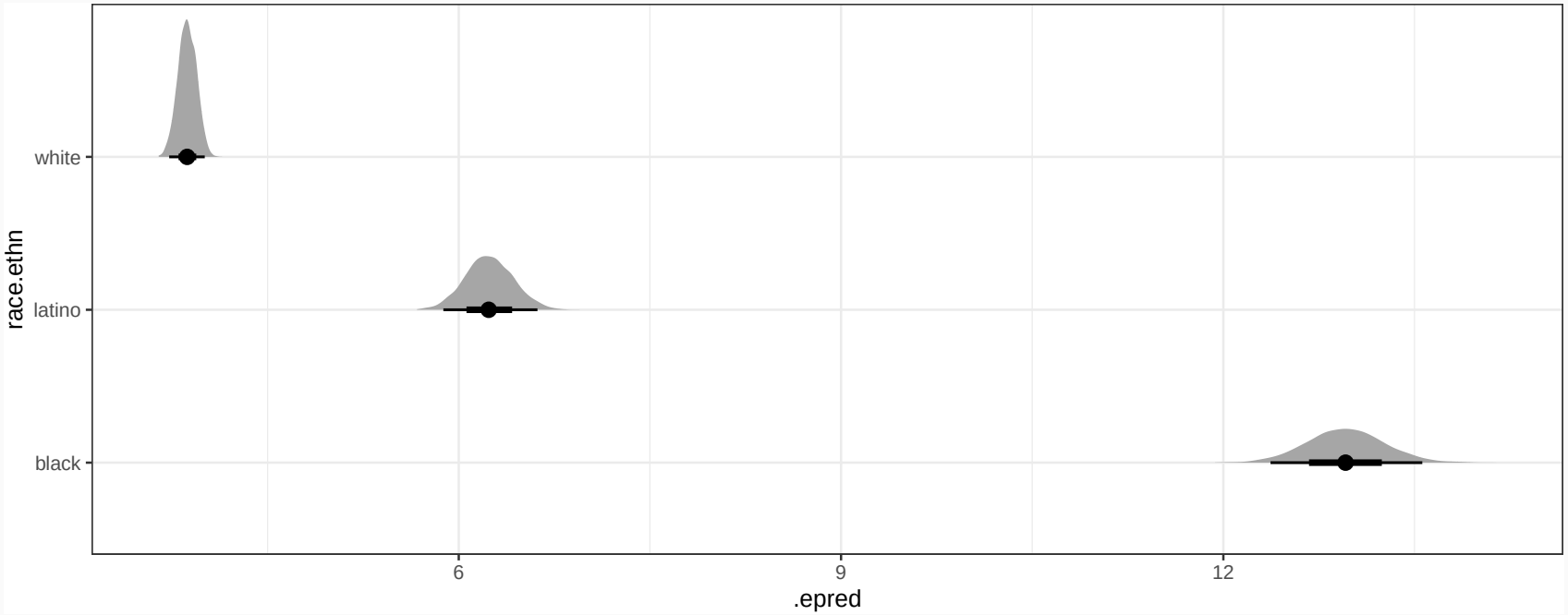
Let's check the expected number of deaths for each group assuming a population of 100,000.

```
sim_dat<-data.frame(race.ethn =
                      c("black", "latino", "white"),
                    pop = 1e5)

sim_dat <- sim_dat |>
  add_epred_draws(fe_pois2)
```

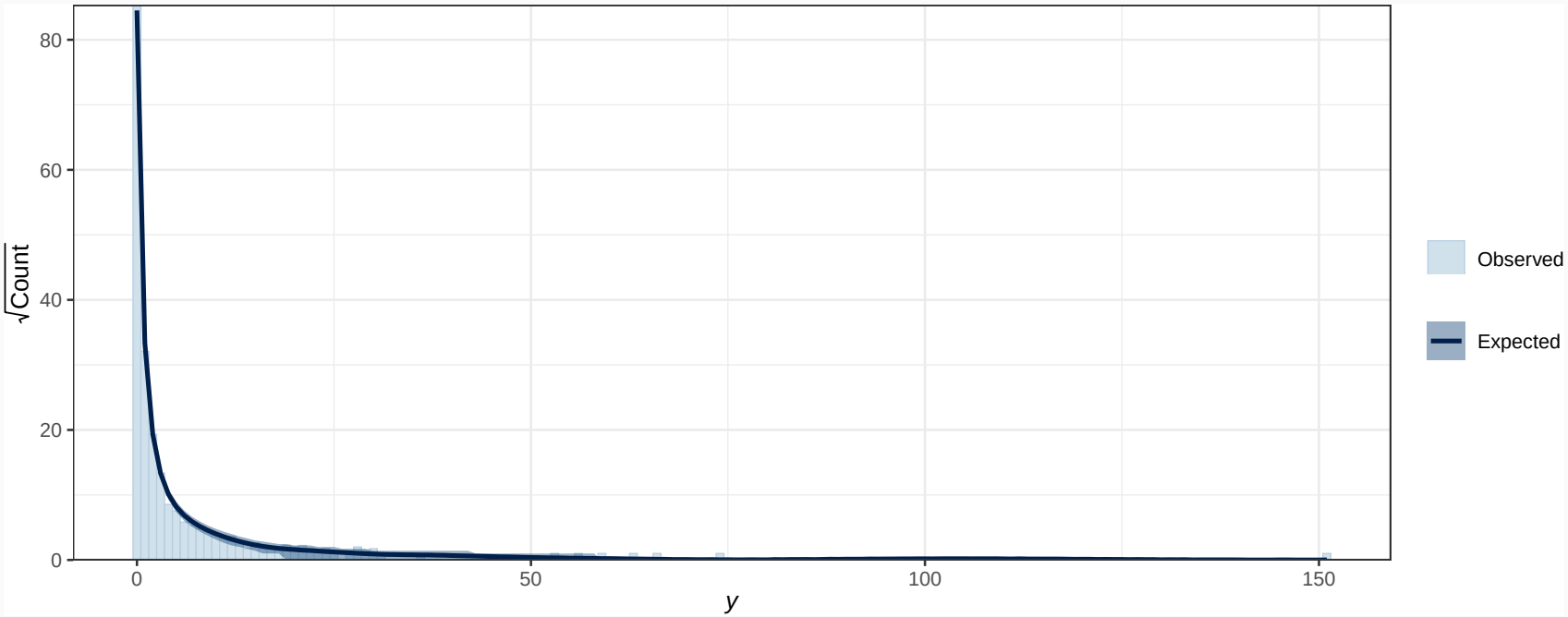
# Posterior inference

```
ggplot(sim_dat,
       aes(x = .epred, y = race.ethn)) +
  stat_slabinterval()
```



# Posterior predictive check

```
pp_check(fe_pois2, type = "rootogram")
```



# Categorical data

Categorical data falls into a fixed set of categories. It may be *unordered*, meaning that there is no inherent ranking of categories, or it may be *ordered*. Ordered categorical data has an explicit hierarchical ranking of values.

# Categorical data, examples

Are these variables ordered or unordered?

# Categorical data, examples

Are these variables ordered or unordered?

- Candidate choice in a primary election

# Categorical data, examples

Are these variables ordered or unordered?

- Candidate choice in a primary election
- Zip code for people choosing a place to move

# Categorical data, examples

Are these variables ordered or unordered?

- Candidate choice in a primary election
- Zip code for people choosing a place to move
- Cause of death

# Categorical data, examples

Are these variables ordered or unordered?

- Candidate choice in a primary election
- Zip code for people choosing a place to move
- Cause of death
- Opinions on a political issue on a thermometer / Likert scale
  (e.g. Strongly oppose, oppose, neutral, support, strongly support)

# Categorical data, examples

Are these variables ordered or unordered?

- Candidate choice in a primary election
- Zip code for people choosing a place to move
- Cause of death
- Opinions on a political issue on a thermometer / Likert scale (e.g. Strongly oppose, oppose, neutral, support, strongly support)
- Graduate program to attend

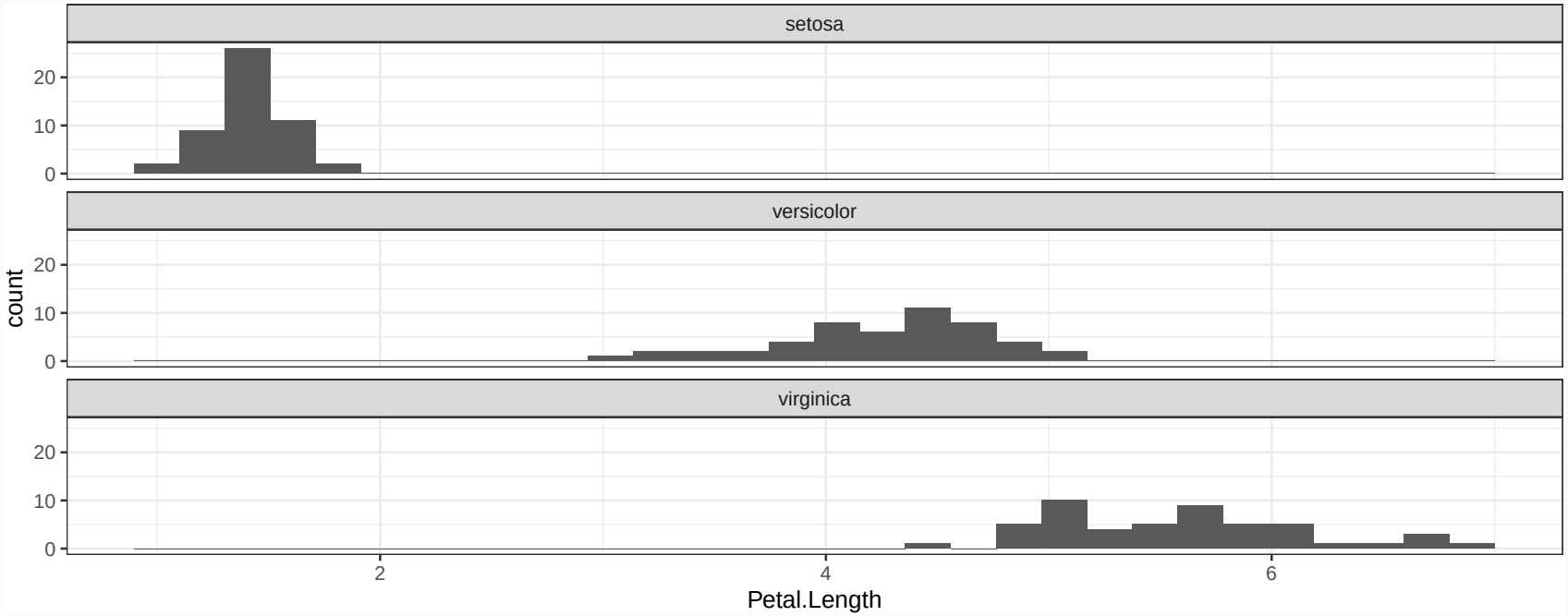
# Categorical data, examples

Are these variables ordered or unordered?

- Candidate choice in a primary election
- Zip code for people choosing a place to move
- Cause of death
- Opinions on a political issue on a thermometer / Likert scale (e.g. Strongly oppose, oppose, neutral, support, strongly support)
- Graduate program to attend
- Ranking of graduate program

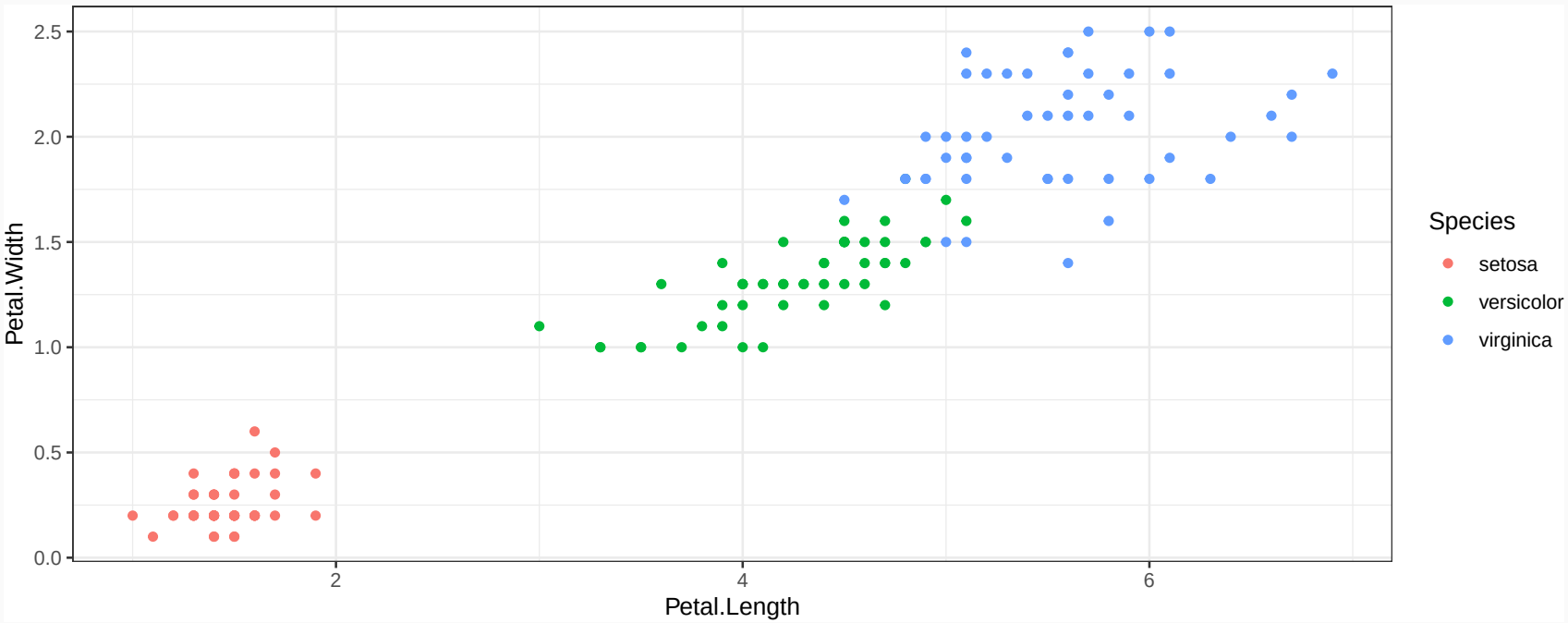
# Visualizing categorical data, facets

```
ggplot(iris, aes(x = Petal.Length)) +
  geom_histogram() +
  facet_wrap(~Species, ncol=1)
```



# Visualizing categorical data, color

```
ggplot(iris, aes(x = Petal.Length, y = Petal.Width,
                 color = Species)) +
  geom_point()
```



# Predicting Iris species from petal length and width

Can we successfully classify Iris species based on the observed length of the petals?

We will use a *multinomial regression*, which is conceptually similar to running $k - 1$ logistic regressions, where $k$ is the number of categories in our outcome.

# Multinomial regression: basics

For a categorical outcome with $K$ categories, estimate $K-1$ models where 1,2,3 stand in for membership in group 1, 2, 3:

$$log\frac{Pr(y_i=1)}{Pr(y_i=K)} = \beta x_i$$

$$log\frac{Pr(y_i=2)}{Pr(y_i=K)} = \beta x_i$$

$$\dots$$

$$log\frac{Pr(y_i=K-1)}{Pr(y_i=K)} = \beta x_i$$

# Implementation

```
iris_model<-brm(Species ~ Petal.Length,
                family = categorical,
                data = iris,
                cores = 4)
```

# These are hard to interpret...

```
summary(iris_model)
```

```
 Family: categorical
  Links: muversicolor = logit; muvirginica = logit
Formula: Species ~ Petal.Length
   Data: iris (Number of observations: 150)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
muversicolor_Intercept      -26.90     12.03   -56.21   -10.73 1.00      864
muvirginica_Intercept       -72.35     18.54  -114.54   -43.65 1.00      843
muversicolor_Petal.Length     9.60      3.83     4.11    18.95 1.00      759
muvirginica_Petal.Length     18.95      5.12    11.29    30.85 1.00      782
                          Tail_ESS
muversicolor_Intercept        1299
muvirginica_Intercept         1260
muversicolor_Petal.Length     1153
muvirginica_Petal.Length      1163

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

# But visualization is helpful

```
conditional_effects(iris_model, categorical = T)
```

